Project - Low Level Design

On

Agriculture Support Triage Agent

Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	DEEPAK PATRO	EN22CS301316
2.	BHOOMIKA PATIDAR	EN22CS301270
3.	CHARU NARIYA	EN22CS301290
4.	DEV KUMAR DAS	EN22CS301321
5.	AVICHAL MISHRA	EN22CS301243

Group Name:07D3

Project Number:AAI-31

Industry Mentor Name:

University Mentor Name:Prof. Maya Yadav

Academic Year:2025-2026

Table of Contents

1. Introduction

The *AI Agriculture Support System* is a multi-agent AI application designed to assist farmers in making better agricultural decisions. The system provides guidance on crop selection, disease diagnosis, and government schemes using artificial intelligence.

The system uses *CrewAI agents, RAG-based knowledge retrieval, and a Streamlit user interface* to generate reliable agricultural recommendations.

2. Scope of the Document

This document provides the *Low Level Design (LLD)* of the AI Agriculture Support System.

It describes:

* System architecture

* Module design

* Process flow

* Information flow

* Component design

* Key design considerations

3. Intended Audience

This document is intended for:

* Project developers

* System architects

* University mentors

* Industry mentors

* Project reviewers

4. System Overview

The AI Agriculture Support System is an intelligent assistant that helps farmers by providing

agricultural guidance through a *multi-agent AI architecture*.

The system includes three main AI agents:

* **Crop Advisor Agent** – provides crop recommendations and farming practices.

* **Disease Expert Agent**– identifies crop diseases and suggests treatments.

* **Policy Advisor Agent** – provides information about government schemes for farmers.

The system also uses *Retrieval Augmented Generation (RAG)* to fetch agricultural knowledge from a document dataset.

5. System Design

The system consists of the following layers:

1. Presentation Layer

* Streamlit UI

* Accepts farmer queries

* Displays AI responses

2. Application Layer

* CrewAI controller

* Manages agents and tasks

3. AI Agent Layer

* Crop Advisor Agent

* Disease Expert Agent

* Policy Advisor Agent

4. Knowledge Layer

* RAG retriever

* Vector database

* Agricultural dataset

5. AI Service Layer

* Groq LLM model for generating responses

6. Application Design

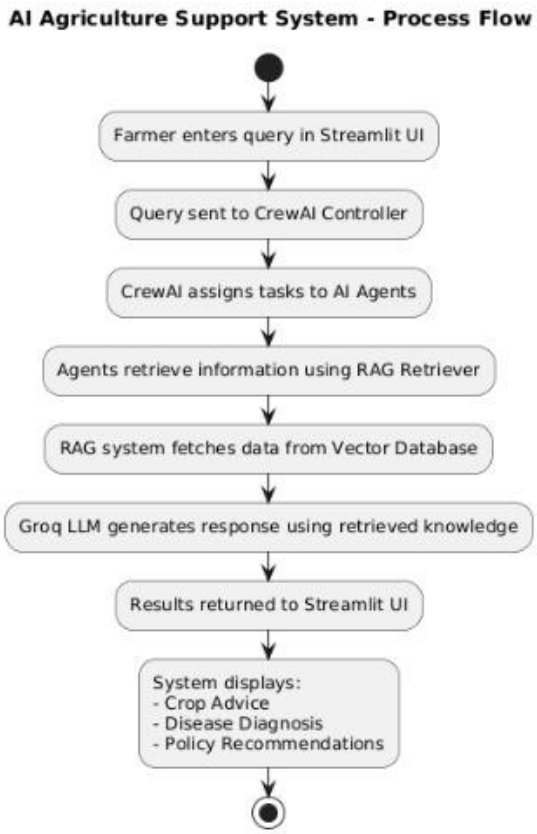
The application is implemented using *Python-based modular architecture*.

Main modules:

Module	Description
App.py	Stremlit user interface
Agents.py	Defines AI agents
Tasks.py	Defines tasks for agents
Crew_setup.py	Controls agent workflow
Rag_setup.py	Creates RAG knowledge system
Data_collector.py	Collects agricultural data

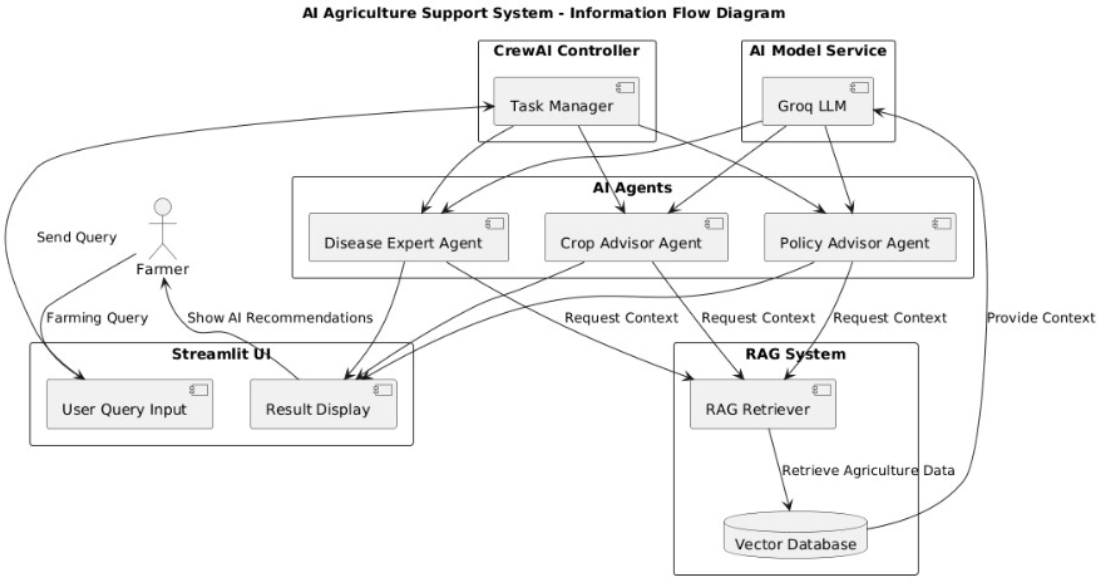
7. Process Flow

The system follows the following process:



8. Information Flow

Information flows through the system in the following sequence:



9. Components Design

1. Streamlit UI

* Accept farmer queries

* Display AI responses

* Provide user-friendly interface

2. CrewAI Controller

* Manage AI agents

* Assign tasks

* Coordinate workflow

3. Crop Advisor Agent

* Suggest suitable crops

* Provide farming practices

* Recommend fertilizers and irrigation

4. Disease Expert Agent

* Identify crop diseases

* Suggest treatment methods

* Provide prevention advice

5. Policy Advisor Agent

* Suggest government schemes

* Provide policy information for farmers

6. RAG Retriever

* Retrieve agricultural information

* Provide context to LLM

7. Vector Database

Stores agricultural documents converted into embeddings.

10. Key Design Considerations

1. Accuracy: RAG ensures responses are based on real agricultural knowledge.

2. Scalability: New agents can be added such as:

* Weather Advisor

* Soil Analysis Agent

3. Usability: The Streamlit interface is simple and easy to use for farmers.

4. Maintainability: The system uses modular Python architecture.

11. API Catalogue

API	Purpose
Groq API	Provides LLM capabilities
Streamlit	Provides web interface
Langchain	Supports RAG retrieval
Crew AI	Multi-agent orchestration

12. References

1. CrewAI Documentation

2. LangChain Documentation

3. Groq LLM API Documentation

4. Streamlit Documentation